

BLUA — Sistema de Gerenciamento de Fila de Atendimento

Relatório Técnico | Sprint: Estruturas de Dados Aplicadas à Saúde Digital
Care Plus | Linguagem: C | Paradigma: Imperativo / Procedural

1. Introdução e Contextualização

A plataforma **Blua** da Care Plus oferece telessaúde proativa, conectando pacientes a médicos remotamente. O desafio central deste sprint foi implementar um **sistema de gerenciamento de fila de atendimento** capaz de: (i) enfileirar pacientes respeitando ordem de chegada (FIFO); (ii) priorizar emergências e urgências; (iii) registrar histórico auditável das consultas finalizadas (LIFO); e (iv) permitir desfazer ações administrativas via pilha de ações. Todas as estruturas foram construídas com **listas encadeadas em C**, sem uso de bibliotecas prontas.

2. Escolha das Estruturas de Dados

Estrutura	Tipo	Implementação	Papel no Sistema
Fila Normal (NORMAL)	FIFO	Lista encadeada simples	Teleconsultas de rotina
Fila Urgente (URGENTE)	FIFO	Lista encadeada simples	Consultas com urgência clínica
Fila Emergência (EMERGENCIA)	FIFO	Lista encadeada simples	Risco de vida imediato
Pilha Histórico (HISTORICO)	LIFO	Lista encadeada simples	Auditoria das últimas 50 consultas
Pilha de Ações (PilhaAcoes)	LIFO	Lista encadeada simples	Undo de cancelamentos e reatribuições

Justificativa: A Fila (FIFO) garante que pacientes sejam atendidos na ordem de chegada dentro de cada nível de prioridade — princípio fundamental da equidade em saúde. A estrutura de *três filas independentes* com despacho hierárquico (emergência → urgente → normal) é mais simples de implementar e auditar do que uma fila de prioridade com heap, mantendo as garantias de ordem. A Pilha (LIFO) é ideal para o histórico de auditoria (acesso recente primeiro) e para undo/redo, onde a última ação realizada é a primeira a ser desfeita.

3. Diagrama de Funcionamento

FILA EMERGÊNCIA (FIFO)		PILHA HISTÓRICO (LIFO)
[frente] → P1 → P2 → [traseira]	→ dequeue →	[topo] ← P_n ← ... ← P1
FILA URGENTE (FIFO)		
[frente] → P3 → P4 → [traseira]	→ dequeue →	
FILA NORMAL (FIFO)		
[frente] → P5 → P6 → [traseira]	→ dequeue →	

Fluxo de despacho: O médico chama `sistema_atender()`. O sistema verifica primeiro a fila EMERGÊNCIA; se vazia, verifica URGENTE; se vazia, consulta NORMAL. O paciente removido (dequeue) é imediatamente empilhado (push) no histórico LIFO para auditoria.

4. Análise de Complexidade

Operação	Fila (FIFO)	Pilha (LIFO)	Observação
enqueue / push	O(1)	O(1)	Inserção sempre no fim/topo — ponteiro direto
dequeue / pop	O(1)	O(1)	Remoção sempre do início/topo — ponteiro direto
peek	O(1)	O(1)	Acesso via ponteiro frente/topo sem remoção
is_empty	O(1)	O(1)	Verifica ponteiro frente/topo == NULL
sistema_atender	O(1)	O(1)	3 checagens fixas de filas + 1 push
sistema_desfazer	O(1)*	O(1)	*reinserção na frente em O(1)
imprimir fila/pilha	O(N)	O(N)	Percorre todos os nós

Uso de memória: O(N) total — cada elemento aloca exatamente um nó na heap via *malloc*. Não há realocações em bloco. A pilha de histórico impõe capacidade máxima de 50 entradas (HISTORICO_MAX), descartando o registro mais antigo ao atingir o limite.

5. Cenários de Uso Demonstrados

#	Nome	Estruturas Exercitadas	Resultado Esperado
1	Fluxo FIFO Básico	Fila NORMAL enqueue + dequeue Pilha HISTÓRICO push	4 pacientes enfileirados; 3 atendidos em ordem de chegada; Histórico LIFO mostra Carla no topo
2	Filas Prioritárias	3 filas (EMERGENCIA, URGENTE, NORMAL) despacho hierárquico	Fernanda (emergência) atendida primeiro, depois Gustavo e Helena (urgentes), depois normais
3	Cancelamento e Desfazer	Pilha de Ações (push/pop) Reinserção FIFO	Kaio cancelado; Pilha de Ações registra; undo reinsere Kaio no início da fila URGENTE

6. Conclusão

A solução implementada atende integralmente aos requisitos do sprint: três filas FIFO independentes com despacho hierárquico garantem priorização de emergências; a pilha LIFO provê histórico auditável com acesso O(1) à consulta mais recente; a pilha de ações permite rastreabilidade e reversibilidade de operações administrativas. Toda a lógica foi construída sobre listas encadeadas alocadas dinamicamente, sem bibliotecas externas, demonstrando domínio das estruturas em nível de implementação. O menu interativo permite validar cada funcionalidade individualmente ou via cenários pré-configurados, facilitando demonstração e auditoria do comportamento do sistema.

Compilação: gcc -o blua src/blua.c | Execução: ./blua | Linguagem: C (C99) | Plataforma: Linux / macOS / Windows (MinGW)